Musterlösung: Strukturierte Daten transformieren und visualisieren

Am Beispiel von Tagesschau-Nachrichten

Prof. Dr. Nicolas Meseth

Inhaltsverzeichnis

Teil 1: Variablen auswählen und Beobachtungen filtern . . . 2
Teil 2: Neue Variablen erzeugen mit `mutate` und `transmute` . . . 5
Teil 3: Daten gruppieren und zusammenfassen . . . 9
Teil 4: Häufigkeiten und Verteilungen visualisieren . . . 13
Teil 5: Gruppenvergleiche visualisieren . . . 17
Teil 6: Zeitliche Entwicklungen visualisieren . . . 21
Teil 7: Zusammenhänge und mehrdimensionale Analysen . . . 27
Teil 8: Datensätze verknüpfen mit Joins . . . 30
Reflexion . . . 38


```r
library(tidyverse)
library(janitor)
library(skimr)
library(ggridges)

articles <- read_csv(
  "data/tagesschau_extracted/articles.csv",
  show_col_types = FALSE
)
```

Teil 1: Variablen auswählen und Beobachtungen filtern

Aufgabe 6

```r
articles_slim <- articles |>
  select(title, date_time, ressort, tag, author, word_count)

nrow(articles_slim)
```

```
[1] 62817
```

Der reduzierte Datensatz `articles_slim` enthält **62.817 Zeilen** — genauso viele wie der ursprüngliche Datensatz, denn `select()` filtert nur Spalten, nie Zeilen.

Aufgabe 7

```r
articles_inland <- articles_slim |>
  filter(ressort == "inland")

nrow(articles_inland)
```

```
[1] 14207
```

Der Datensatz enthält **14.207 Inland-Artikel**.

Aufgabe 8

```r
articles_ausland_lang <- articles_slim |>
  filter(
    ressort == "ausland",
    date_time >= as.Date("2020-01-01"),
    word_count > 800
  )

nrow(articles_ausland_lang)
```

```
[1] 1316
```

```r
articles_ausland_lang |>
  arrange(date_time) |>
  slice(1) |>
  select(title, date_time, word_count)
```

```
# A tibble: 1 x 3
  title                                           date_time           word_count
  <chr>                                           <dttm>                   <dbl>
1 Krise USA und Iran: EU und NATO starten Aktivi~ 2020-01-07 02:33:11        851
```

Es gibt **1.316 solcher Artikel**. Der älteste datiert vom **7. Januar 2020**.

Aufgabe 9

```r
# Anteil der Artikel ohne Autorenangabe
na_author_pct <- round(
  100 * sum(is.na(articles_slim$author)) / nrow(articles_slim),
  1
)
na_author_pct
```

```
[1] 26.6
```

```r
articles_with_author <- articles_slim |>
  filter(!is.na(author))

nrow(articles_with_author)
```

```
[1] 46083
```

26,6 % der Artikel — gut ein Viertel — haben keine Autorenangabe. Das verbleibende Objekt `articles_with_author` hat **46.083 Zeilen**.

Ein fehlender `author`-Wert bedeutet hier nicht „unbekannt", sondern sehr wahrscheinlich „kein namentlicher Autor angegeben". Das ist semantisch ein Unterschied: Es handelt sich nicht um eine Datenlücke, sondern um eine bewusste Nicht-Angabe. Viele Artikel bei tagesschau.de werden redaktionell ohne Namensnennung veröffentlicht und laufen unter `"tagesschau.de"` als Kollektivautor — oder eben ganz ohne Eintrag.

Aufgabe 10

```r
articles_slim |>
  arrange(desc(word_count)) |>
  slice_max(word_count, n = 10) |>
  select(title, ressort, word_count)
```

```
# A tibble: 10 x 3
   title                                                    ressort word_count
   <chr>                                                    <chr>        <dbl>
 1 "Chronik zum NSU: Rassistische Morde, staatliches Versage~ inland       11165
 2 "Iran-Krieg: ++ NATO erhöht Einsatzbereitschaft der Raket~ newsti~       9131
 3 "ARD-faktenfinder: Faktenchecks zur Corona-Pandemie"       fakten~       9053
 4 "Liveblog: ++ Papst starb in Folge eines Schlaganfalls ++" newsti~       8396
 5 "Chronik zu Corona: Ausnahmezustand als neue Normalität"   fakten~       8119
 6 "Liveblog: ++ Ansteckungsrate steigt wieder ++"            newsti~       8058
 7 "Nahost-Liveblog: ++ Iran: Straße von Hormus gesperrt ++"  newsti~       7999
 8 "Ziele der \"Sonstigen\" - Kleinparteien bei der Europawa~ europa~       7892
 9 "Nahost-Liveblog: ++ Guterres fordert Ende aller Kämpfe +~ newsti~       7735
10 "Liveblog zu Trump: ++ Klage gegen Trumps Erlass zur Staa~ newsti~       7661
```

Die längsten Artikel sind durchgängig in den Ressorts **`newsticker`** und **`faktenfinder`** zu finden. Das erklärt sich inhaltlich: Newsticker und Faktenchecks werden kontinuierlich um neue Absätze erweitert, ohne dass ein neuer Artikel angelegt wird — sie akkumulieren dadurch Wortanzahlen, die normalen Artikeln nicht entsprechen.

Teil 2: Neue Variablen erzeugen mit `mutate` und `transmute`

Aufgabe 11

```
articles_slim <- articles_slim |>
  mutate(
    year    = year(date_time),
    month   = month(date_time),
    weekday = wday(date_time, label = TRUE)
  )

levels(articles_slim$weekday)
```

```
[1] "So" "Mo" "Di" "Mi" "Do" "Fr" "Sa"
```

Der Wochentag wird als geordneter Faktor mit sieben Stufen zurückgegeben. Die Abkürzungen und Namen hängen von der Systemsprache (Locale) ab — auf einem deutschsprachigen System erscheinen `"So"`, `"Mo"`, …, `"Sa"`, auf einem englischsprachigen `"Sun"`, `"Mon"`, …, `"Sat"`. Die Reihenfolge beginnt standardmäßig mit Sonntag.

Aufgabe 12

```
articles_slim <- articles_slim |>
  mutate(
    ressort = as_factor(ressort),
    tag     = as_factor(tag)
  )

nlevels(articles_slim$ressort)
```

```
[1] 39
```

```r
nlevels(articles_slim$tag)
```

```
[1] 383
```

```r
levels(articles_slim$ressort) |> head(15)
```

```
 [1] "inland"
 [2] "ausland"
 [3] "wirtschaft"
 [4] "Einstellungen einblenden Pfeil rechts"
 [5] "kultur"
 [6] "schlusslicht"
 [7] "wissen"
 [8] "hilfe"
 [9] "korrespondenten"
[10] "multimedia"
[11] "kommentar"
[12] "jahresrueckblick"
[13] "europawahl"
[14] "wetterthema"
[15] "eilmeldung"
```

Das Ressort hat **39 Ausprägungen**, der Tag **383**. Auffällig: Viele Ressort-Ausprägungen sind offensichtlich keine redaktionellen Ressorts, sondern technische Artefakte (z. B. `"Einstellungen einblenden Pfeil rechts"`). Auch bei den Tags gibt es Duplikate wie `"FAKTENFINDER"` und `"#FAKTENFINDER"` — hier besteht Bereinigungsbedarf.

Aufgabe 13

```r
articles_slim <- articles_slim |>
  mutate(
    title_length     = str_length(title),
    title_word_count = str_count(title, " ") + 1
  )
```

```r
summarize(articles_slim,
  min    = min(title_length, na.rm = TRUE),
  median = median(title_length, na.rm = TRUE),
  max    = max(title_length, na.rm = TRUE)
)
```

```
# A tibble: 1 x 3
    min median   max
  <int>  <int> <int>
1     3     55   146
```

```r
# Kürzeste Titel
articles_slim |>
  arrange(title_length) |>
  slice(1:3) |>
  select(title, title_length, ressort)
```

```
# A tibble: 3 x 3
  title    title_length ressort
  <chr>           <int> <fct>
1 CDU                 3 nachrichten
2 Software            8 hilfe
3 Mai 2000            8 multimedia
```

```r
# Längste Titel
articles_slim |>
  arrange(desc(title_length)) |>
  slice(1:3) |>
  select(title, title_length, ressort)
```

```
# A tibble: 3 x 3
  title                                                     title_length ressort
  <chr>                                                            <int> <fct>
1 15 Minuten Investitionsbooster für Unternehmen / Keine s~          146 multim~
2 15 Minuten. Der tagesschau-Podcast am Morgen Eskalation ~          139 multim~
3 15 Minuten. Der tagesschau-Podcast am Morgen Geschäfte m~          138 multim~
```

Der **Median** der Titellänge beträgt **55 Zeichen**. Der kürzeste Titel ist nur 3 Zeichen lang, der längste erreicht 146 Zeichen — ein langer Liveblog-Eintrag. Die meisten Titel bewegen sich im Bereich von 47 bis 61 Zeichen (1. bis 3. Quartil).

Aufgabe 14

```r
articles_timeline <- articles_slim |>
  transmute(
    year,
    month,
    weekday,
    ressort,
    word_count,
    title_length
  )

glimpse(articles_timeline)
```

```
Rows: 62,817
Columns: 6
$ year         <dbl> 2006, 2006, 2006, 2006, 2006, 2006, 2006, 2006, 2006, 200~
$ month        <dbl> 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, ~
$ weekday      <ord> Do, Fr, Fr, Sa, So, Mi, Mi, Do, Do, Di, Do, Di, Mi, Do, D~
$ ressort      <fct> inland, ausland, inland, ausland, wirtschaft, wirtschaft,~
$ word_count   <dbl> 569, 406, 658, 264, 601, 399, 262, 516, 801, 433, 567, 23~
$ title_length <int> 32, 46, 50, 50, 49, 41, 41, 33, 43, 40, 49, 45, 23, 48, 3~
```

`transmute()` ist dann nützlicher als `mutate()`, wenn man gezielt einen neuen, kompakten Datensatz erzeugen möchte, ohne die ursprünglichen Spalten mitzuschleppen. Ein typischer Anwendungsfall: Man möchte einen Datensatz zur Aggregation oder zum Export vorbereiten und benötigt dabei nur die neu berechneten Variablen — nicht den gesamten Ausgangsdatensatz.

Aufgabe 15

```r
articles_slim <- articles_slim |>
  mutate(
    article_type = case_when(
      word_count < 200  ~ "Kurzmeldung",
```

```r
      word_count < 700  ~ "Standardartikel",
      word_count >= 700 ~ "Langer Artikel"
    ),
    article_type = factor(
      article_type,
      levels = c("Kurzmeldung", "Standardartikel", "Langer Artikel")
    )
  )

articles_slim |>
  count(article_type) |>
  mutate(anteil_pct = round(100 * n / sum(n, na.rm = TRUE), 1))
```

```
# A tibble: 4 x 3
  article_type        n anteil_pct
  <fct>           <int>      <dbl>
1 Kurzmeldung      4386        7
2 Standardartikel 44890       71.5
3 Langer Artikel  13174       21
4 <NA>              367        0.6
```

Der mit Abstand häufigste Typ ist der **Standardartikel** (ca. 71,5 % aller Artikel). Nur etwa 7 % sind Kurzmeldungen, und 21 % sind Lange Artikel. Rund 0,6 % der Zeilen haben kein `word_count` und werden zu `NA` ausgewertet — diese entsprechen den fehlenden Wortanzahlen im ursprünglichen Datensatz.

Teil 3: Daten gruppieren und zusammenfassen

Aufgabe 16

```r
articles_slim |>
  count(ressort, sort = TRUE) |>
  print(n = 20)
```

```
# A tibble: 40 x 2
   ressort                                    n
   <fct>                                   <int>
```

```
 1 ausland                                  24061
 2 wirtschaft                               16609
 3 inland                                   14207
 4 wissen                                    2194
 5 investigativ                              1107
 6 faktenfinder                              1083
 7 newsticker                                1053
 8 multimedia                                 758
 9 kommentar                                  502
10 kultur                                     405
11 Einstellungen einblenden Pfeil rechts      260
12 europawahl                                 174
13 eilmeldung                                 131
14 nachrichten                                 88
15 sport                                       65
16 jahresrueckblick                            20
17 wetter                                      14
18 schlusslicht                                 9
19 hilfe                                        8
20 news                                         8
# i 20 more rows
```

Das häufigste Ressort ist **`ausland`** (24.061 Artikel), gefolgt von `wirtschaft` und `inland`. Ressorts mit weniger als 100 Artikeln sind für ressortspezifische Analysen in der Regel wenig geeignet, da die kleinen Fallzahlen zu instabilen Kenngrößen führen.

Die sechs **Hauptressorts** sind: `ausland`, `wirtschaft`, `inland`, `wissen`, `investigativ`, `faktenfinder`.

Aufgabe 17

```r
hauptressorts <- c("inland", "ausland", "wirtschaft", "wissen", "investigativ", "faktenfinde

articles_slim |>
  filter(ressort %in% hauptressorts) |>
  group_by(ressort) |>
  summarize(
    n                = n(),
    median_word_count = median(word_count, na.rm = TRUE),
```

```r
    mean_word_count   = round(mean(word_count, na.rm = TRUE)),
    pct_named_author  = round(
      100 * mean(!is.na(author) & author != "tagesschau.de"),
      1
    )
  ) |>
  arrange(desc(mean_word_count))
```

```
# A tibble: 6 x 5
  ressort          n median_word_count mean_word_count pct_named_author
  <fct>        <int>             <dbl>           <dbl>            <dbl>
1 investigativ  1107               802             797             96.9
2 faktenfinder  1083               764             792             88.1
3 wissen        2194               602             613             68.7
4 inland       14207               524             561             39.6
5 wirtschaft   16609               424             491             30.3
6 ausland      24061               428             463             37.9
```

investigativ produziert mit Abstand die längsten Artikel (Median: 802 Wörter). Fast alle investigativen Artikel tragen einen namentlichen Autor (96,9 %). **ausland** hat die kürzesten Artikel im Mittel (463 Wörter), was gut zur schnellen Nachrichtennatur des Auslands-Ressorts passt.

Aufgabe 18

```r
articles_slim |>
  filter(ressort %in% c("inland", "ausland", "wirtschaft")) |>
  group_by(year, ressort) |>
  summarize(n = n(), .groups = "drop") |>
  pivot_wider(names_from = ressort, values_from = n) |>
  print(n = 25)
```

```
# A tibble: 21 x 4
    year inland ausland wirtschaft
   <dbl>  <int>   <int>      <int>
 1  2006     48     117         39
 2  2007    198     365        275
```

3	2008	88	306	338
4	2009	136	286	832
5	2010	193	223	767
6	2011	198	359	944
7	2012	215	286	995
8	2013	207	361	671
9	2014	174	416	439
10	2015	336	637	680
11	2016	391	835	395
12	2017	319	725	158
13	2018	396	704	206
14	2019	344	904	168
15	2020	752	1103	302
16	2021	735	793	578
17	2022	973	1435	966
18	2023	1840	2754	1802
19	2024	3168	4995	2767
20	2025	2752	5071	2613
21	2026	744	1386	674

Die Jahre **2006** und **2026** sind offensichtlich unvollständige Jahrgänge — 2006 beginnt erst im Januar, 2026 endet zu einem noch unbekannten Zeitpunkt. Beim Vergleich von Jahreswerten sollte man diese Jahrgänge entweder ausschließen oder mit einem entsprechenden Hinweis versehen. Für Zeitreihenanalysen empfiehlt sich ein Filter auf `year >= 2007 & year <= 2025`.

Aufgabe 19

```
articles_mit_median <- articles_slim |>
  filter(ressort %in% hauptressorts) |>
  group_by(ressort) |>
  mutate(
    ressort_median_wc = median(word_count, na.rm = TRUE),
    above_median      = word_count > ressort_median_wc
  ) |>
  ungroup()

articles_mit_median |>
  filter(ressort == "inland") |>
```

```r
  summarize(
    n_gesamt      = n(),
    n_above       = sum(above_median, na.rm = TRUE),
    anteil_above  = round(100 * mean(above_median, na.rm = TRUE), 1)
  )
```

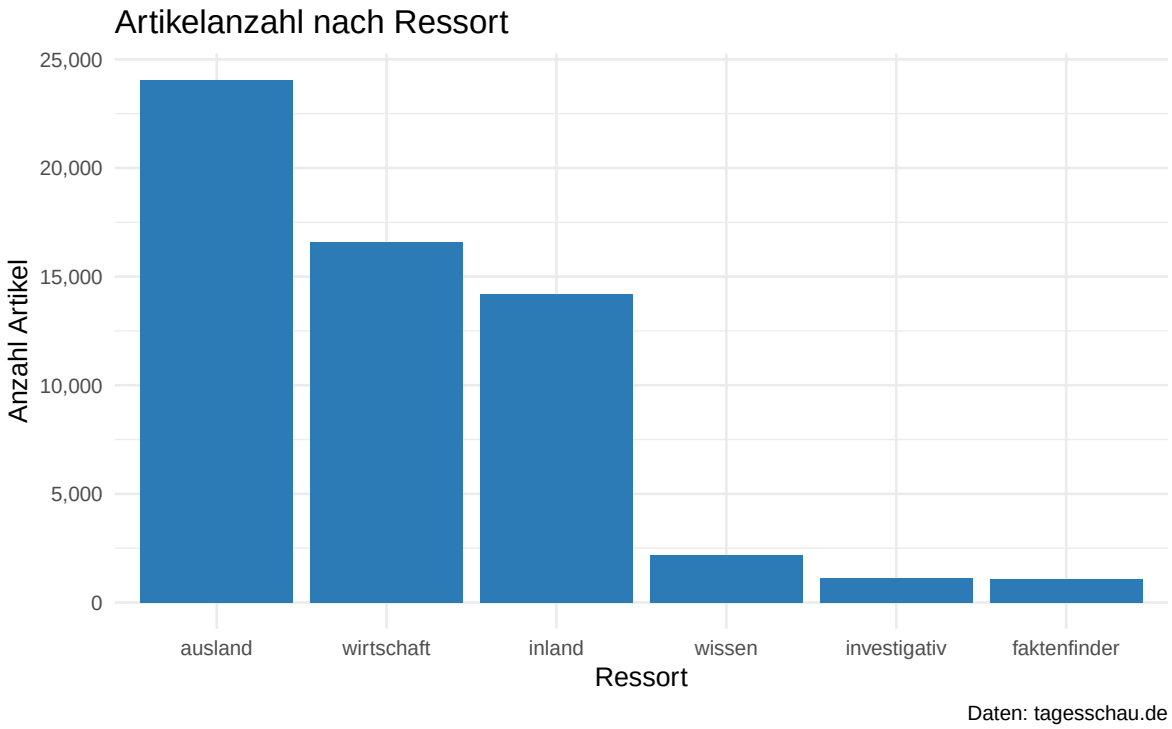
```
# A tibble: 1 x 3
  n_gesamt n_above anteil_above
     <int>   <int>        <dbl>
1    14207    7069           50
```

Von den 14.207 Inland-Artikeln liegen **7.069** (ca. 49,8 %) über dem eigenen Ressort-Median. Das ist zwar nicht überraschend — per Definition liegen bei einer stetigen Verteilung etwa 50 % der Werte über dem Median —, aber es verdeutlicht den zentralen Unterschied zwischen `group_by() + mutate()` und `group_by() + summarize()`: Während `summarize()` den Datensatz auf eine Zeile pro Gruppe reduziert, bleibt die Zeilenanzahl bei `mutate()` vollständig erhalten. Jeder Artikel erhält seinen gruppenspezifischen Medianwert als eigene Spalte.

Teil 4: Häufigkeiten und Verteilungen visualisieren

Aufgabe 20

```r
articles_slim |>
  filter(ressort %in% hauptressorts) |>
  mutate(ressort = fct_infreq(ressort)) |>
  ggplot(aes(x = ressort)) +
  geom_bar(fill = "#2c7bb6") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title   = "Artikelanzahl nach Ressort",
    x       = "Ressort",
    y       = "Anzahl Artikel",
    caption = "Daten: tagesschau.de"
  ) +
  theme_minimal(base_size = 13)
```

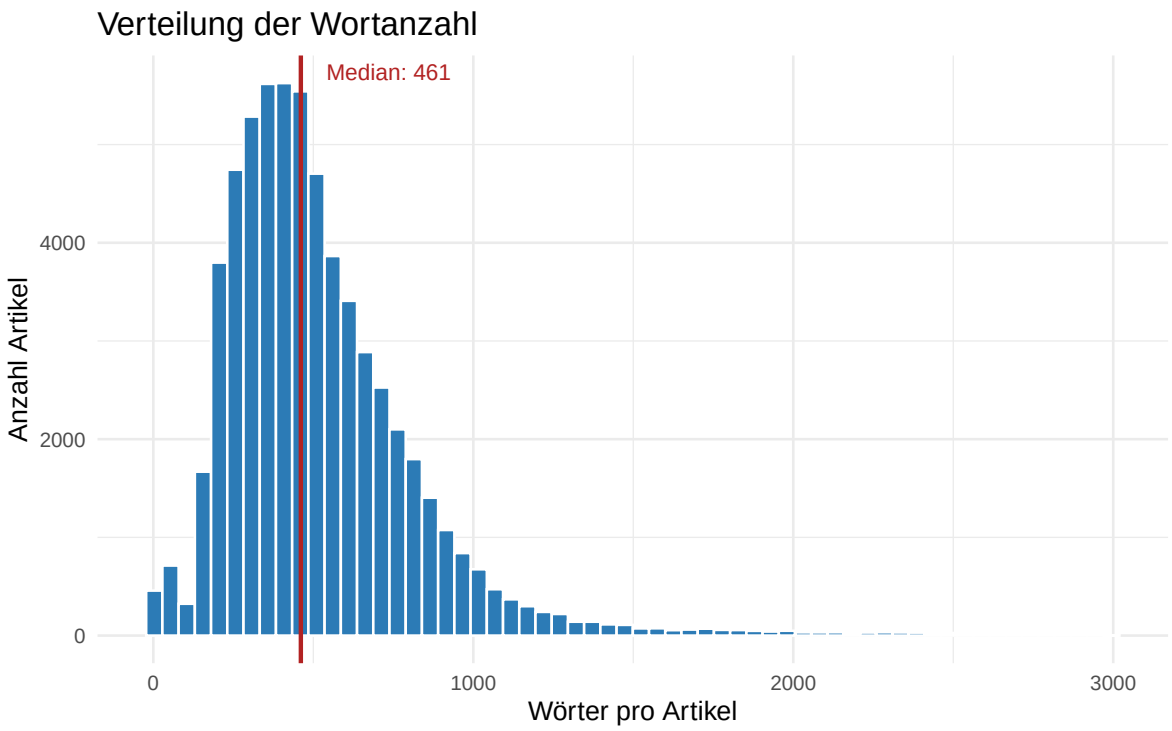


Aufgabe 21

```
median_wc <- median(articles_slim$word_count, na.rm = TRUE)

articles_slim |>
  filter(word_count < 3000) |>
  ggplot(aes(x = word_count)) +
  geom_histogram(bins = 60, fill = "#2c7bb6", color = "white") +
  geom_vline(xintercept = median_wc, color = "firebrick", linewidth = 1) +
  annotate(
    "text",
    x = median_wc + 80, y = Inf,
    label = paste0("Median: ", median_wc),
    hjust = 0, vjust = 1.5, color = "firebrick", size = 4
  ) +
  labs(
    title = "Verteilung der Wortanzahl",
```

```
    x      = "Wörter pro Artikel",
    y      = "Anzahl Artikel"
  ) +
  theme_minimal(base_size = 13)
```

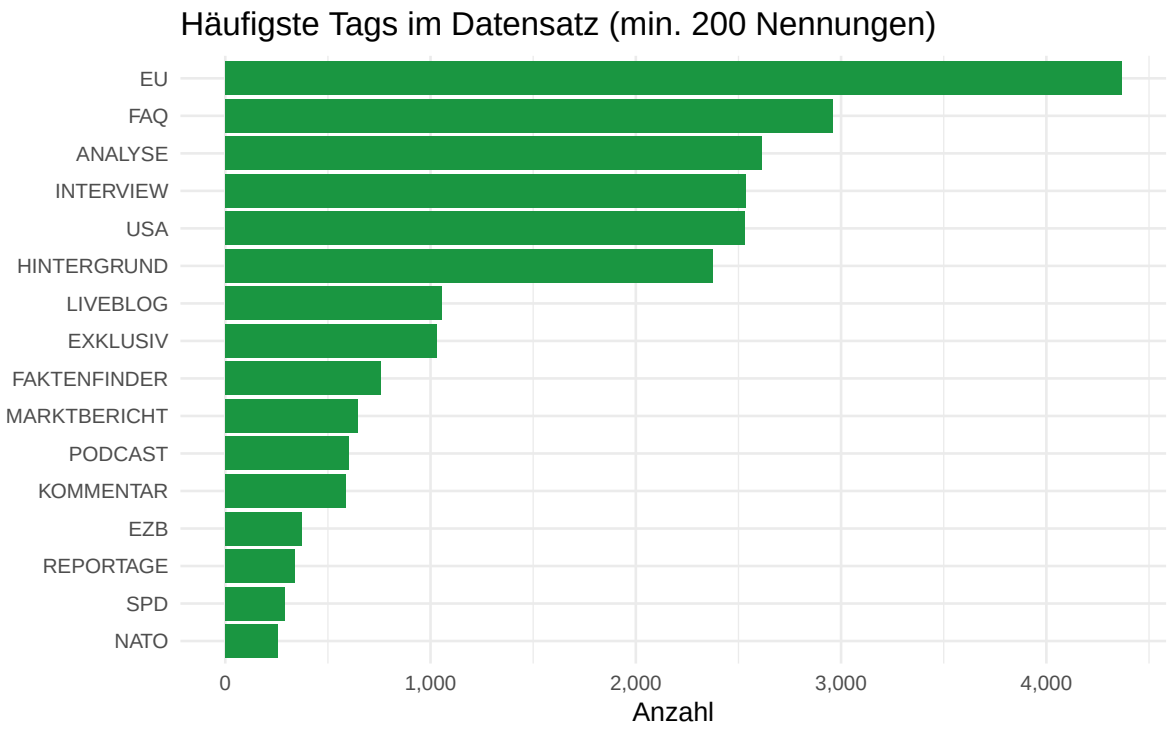


Die Verteilung ist **rechtsschief**: Es gibt viele Artikel mit 200–700 Wörtern und einen langen Schwanz hin zu sehr langen Texten. Der Median liegt bei 461 Wörtern, der Mittelwert ist höher (~538) — ein sicheres Zeichen für Rechtsschieflage. Typische tagesschau.de-Artikel sind also eher mittelkurze Nachrichtentexte.

Aufgabe 22

```
articles_slim |>
  filter(!is.na(tag)) |>
  mutate(tag_clean = str_remove_all(tag, "#") |> str_to_upper()) |>
  count(tag_clean, sort = TRUE) |>
  filter(n >= 200) |>
```

```r
mutate(tag_clean = fct_reorder(tag_clean, n)) |>
ggplot(aes(x = tag_clean, y = n)) +
geom_col(fill = "#1a9641") +
coord_flip() +
scale_y_continuous(labels = scales::comma) +
labs(
  title = "Häufigste Tags im Datensatz (min. 200 Nennungen)",
  x     = NULL,
  y     = "Anzahl"
) +
theme_minimal(base_size = 13)
```

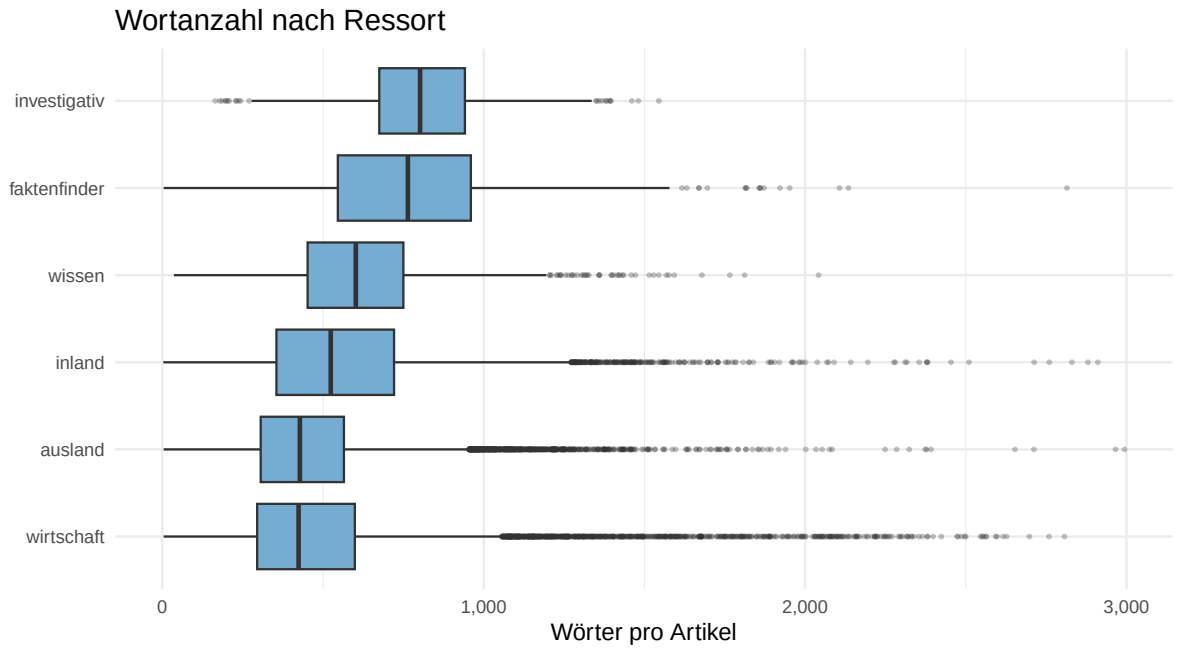


Der häufigste Tag ist **EU** (>4.300 Mal), gefolgt von **FAQ**, **ANALYSE** und **INTERVIEW**. Inhaltlich zeigen die Top-Tags, dass tagesschau.de stark europapolitisch geprägt ist. Der hohe FAQ-Anteil deutet auf journalistische Erklärformate hin, die in den letzten Jahren stark zugenommen haben.

Teil 5: Gruppenvergleiche visualisieren

Aufgabe 23

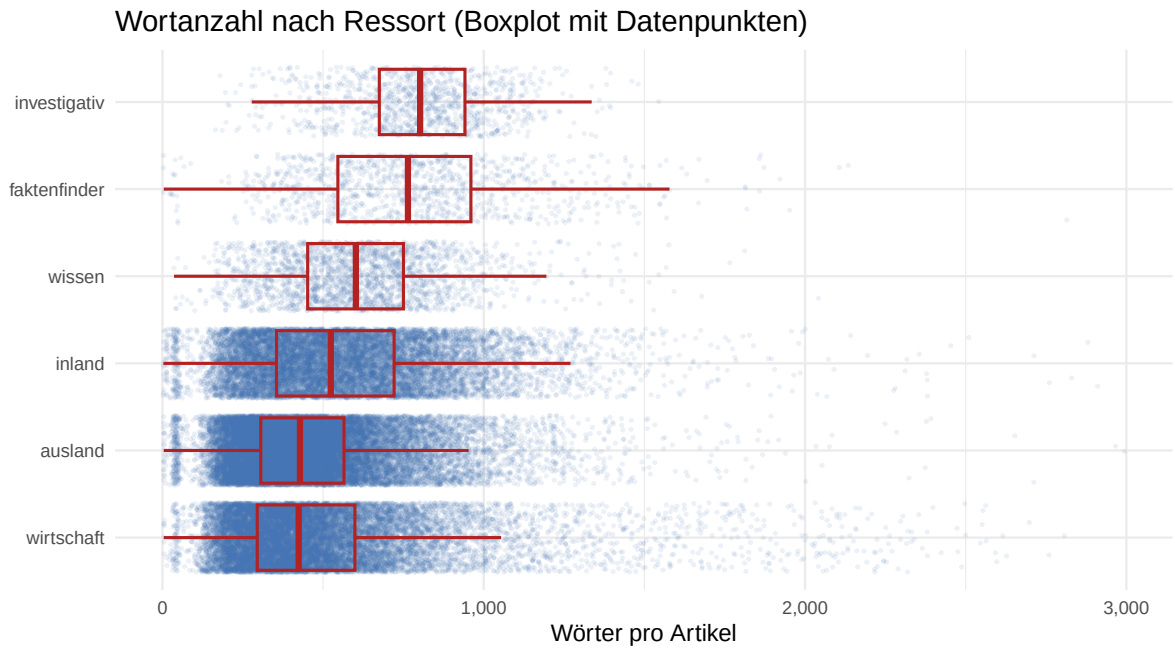
```
articles_slim |>
  filter(ressort %in% hauptressorts, word_count < 3000) |>
  mutate(ressort = fct_reorder(ressort, word_count, .fun = median)) |>
  ggplot(aes(x = ressort, y = word_count)) +
  geom_boxplot(fill = "#74add1", outlier.alpha = 0.3, outlier.size = 0.8) +
  coord_flip() +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Wortanzahl nach Ressort",
    x     = NULL,
    y     = "Wörter pro Artikel"
  ) +
  theme_minimal(base_size = 13)
```



`investigativ` hat die größte Streuung und den höchsten Median. Das spiegelt den journalistischen Charakter wider: Investigative Recherchen sind detaillierte, aufwändige Texte. **`wirtschaft`** zeigt trotz hoher Fallzahl eine vergleichsweise moderate Streuung — viele Wirtschaftsartikel folgen dem Standardformat einer Nachrichtenmeldung.

Aufgabe 24

```r
articles_slim |>
  filter(ressort %in% hauptressorts, word_count < 3000) |>
  mutate(ressort = fct_reorder(ressort, word_count, .fun = median)) |>
  ggplot(aes(x = ressort, y = word_count)) +
  geom_jitter(alpha = 0.1, size = 0.5, color = "#4575b4") +
  geom_boxplot(fill = NA, color = "firebrick", linewidth = 0.8, outlier.shape = NA) +
  coord_flip() +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Wortanzahl nach Ressort (Boxplot mit Datenpunkten)",
    x     = NULL,
    y     = "Wörter pro Artikel"
  ) +
  theme_minimal(base_size = 13)
```

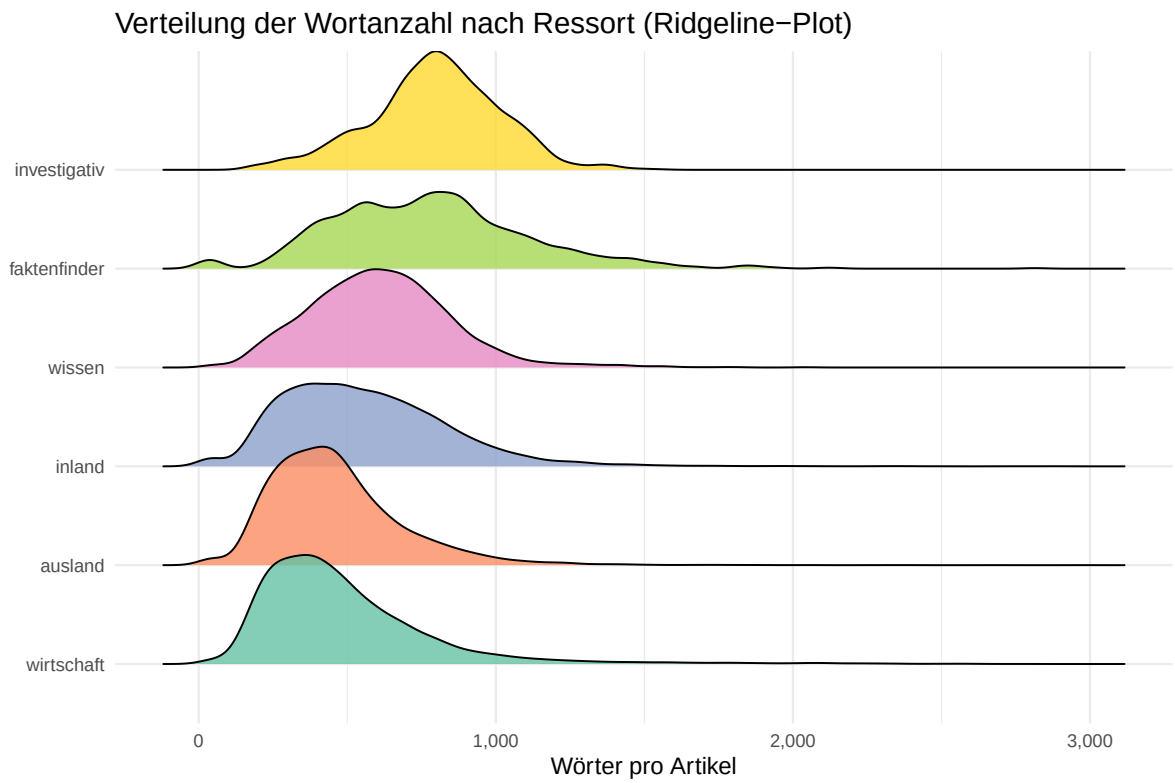


Bei Ressorts mit zehntausenden Artikeln (wie `ausland` oder `wirtschaft`) entsteht durch `geom_jitter()` eine dichte Punktewolke, die kaum noch individuelle Punkte erkennen lässt.

Die Stärke dieser kombinierten Darstellung liegt darin, dass man **sowohl** die Verteilungszusammenfassung (Boxplot) **als auch** die tatsächliche Datendichte (Jitter) sieht — Ausreißer und Häufungen werden deutlicher als beim reinen Boxplot.

Aufgabe 25

```r
articles_slim |>
  filter(ressort %in% hauptressorts, word_count < 3000) |>
  mutate(ressort = fct_reorder(ressort, word_count, .fun = median)) |>
  ggplot(aes(x = word_count, y = ressort, fill = ressort)) +
  geom_density_ridges(alpha = 0.8, scale = 1.2) +
  scale_fill_brewer(palette = "Set2", guide = "none") +
  scale_x_continuous(labels = scales::comma) +
  labs(
    title = "Verteilung der Wortanzahl nach Ressort (Ridgeline-Plot)",
    x     = "Wörter pro Artikel",
    y     = NULL
  ) +
  theme_minimal(base_size = 13)
```

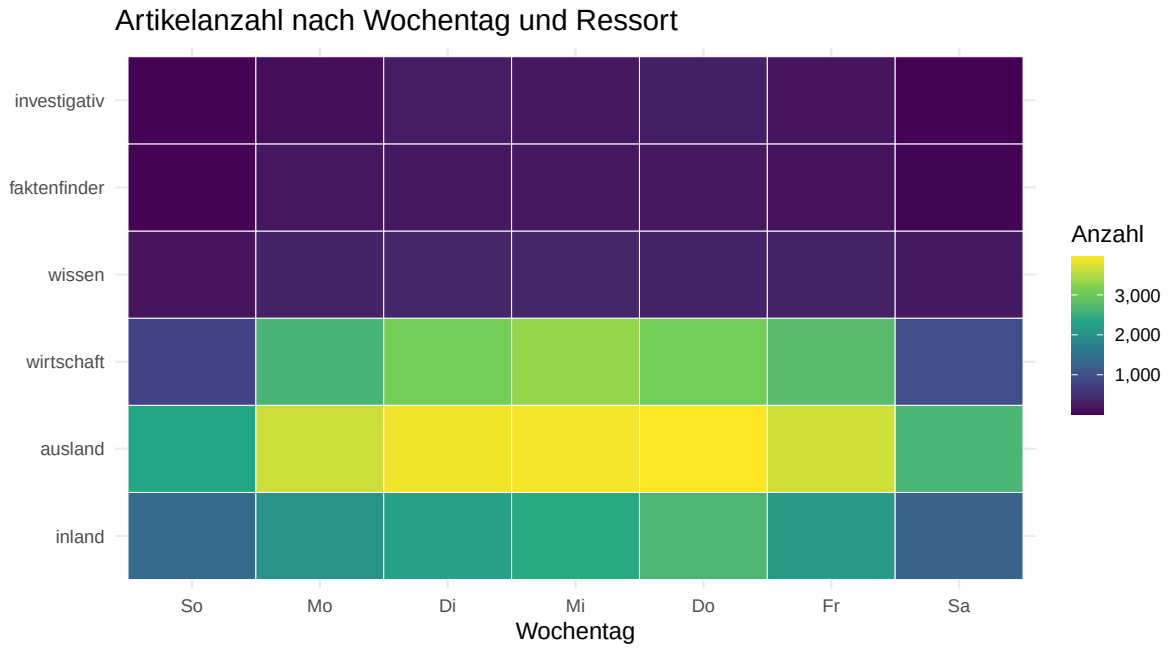


Der **Boxplot** zeigt kompakt Median, Quartile und Ausreißer, verbirgt aber die genaue Form der Verteilung — ob sie z. B. bimodal oder schief ist. Der **Ridgeline-Plot** macht die vollständige Dichteverteilung sichtbar: Hier erkennt man z. B. die unterschiedliche Kurvensteilheit bei `wissen` gegenüber `ausland`. Dafür ist er weniger geeignet, wenn es um exakte Vergleiche von Median oder Quartilsabständen geht. Für explorative Zwecke mit vielen Gruppen ist der Ridgeline-Plot oft der aufschlussreichere erste Blick.

Aufgabe 26

```
articles_slim |>
  filter(ressort %in% hauptressorts) |>
  mutate(weekday = wday(date_time, label = TRUE)) |>
  group_by(weekday, ressort) |>
  summarize(n = n(), .groups = "drop") |>
  ggplot(aes(x = weekday, y = ressort, fill = n)) +
  geom_tile(color = "white") +
```

```r
  scale_fill_viridis_c(labels = scales::comma, option = "D") +
  labs(
    title = "Artikelanzahl nach Wochentag und Ressort",
    x     = "Wochentag",
    y     = NULL,
    fill  = "Anzahl"
  ) +
  theme_minimal(base_size = 13)
```



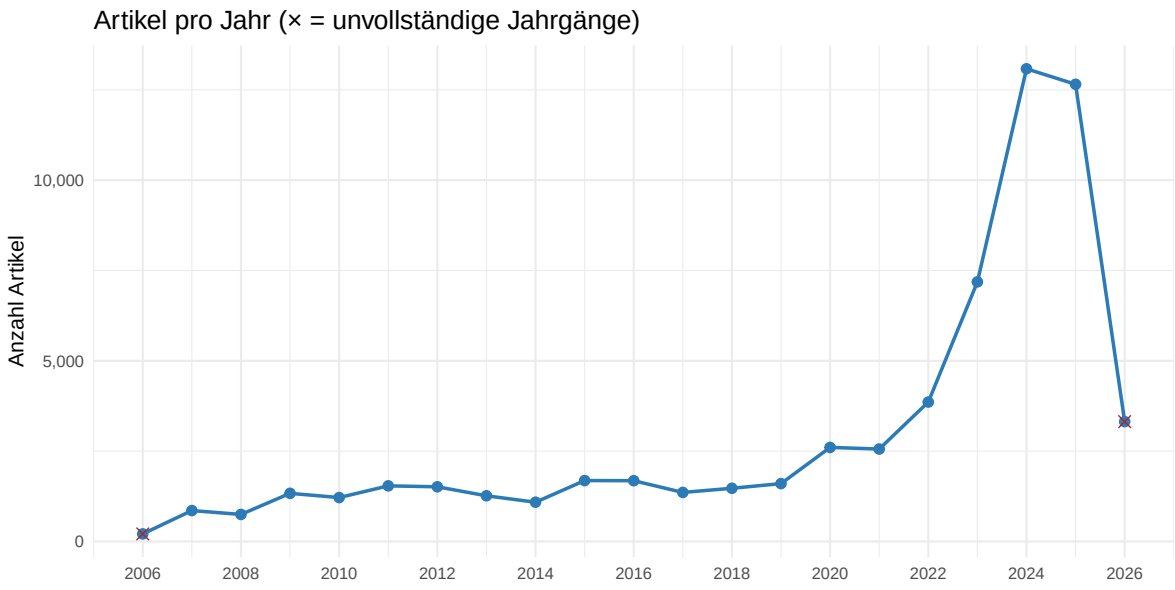
Am Wochenende (Samstag und Sonntag) erscheinen deutlich weniger Artikel — am stärksten sichtbar bei `wirtschaft`, `investigativ` und `faktenfinder`. Das `ausland`-Ressort ist vergleichsweise wochenendresistent, da internationale Ereignisse keine Pause kennen. Die stärksten Werktage sind Dienstag bis Donnerstag.

Teil 6: Zeitliche Entwicklungen visualisieren

Aufgabe 27

```r
articles_per_year <- articles_slim |>
  count(year)

articles_per_year |>
  ggplot(aes(x = year, y = n)) +
  geom_line(color = "#2c7bb6", linewidth = 1) +
  geom_point(color = "#2c7bb6", size = 2.5) +
  # unvollständige Jahrgänge markieren
  geom_point(
    data = filter(articles_per_year, year %in% c(2006, 2026)),
    color = "firebrick", size = 3, shape = 4
  ) +
  scale_y_continuous(labels = scales::comma) +
  scale_x_continuous(breaks = seq(2006, 2026, 2)) +
  labs(
    title   = "Artikel pro Jahr (× = unvollständige Jahrgänge)",
    x       = NULL,
    y       = "Anzahl Artikel"
  ) +
  theme_minimal(base_size = 13)
```

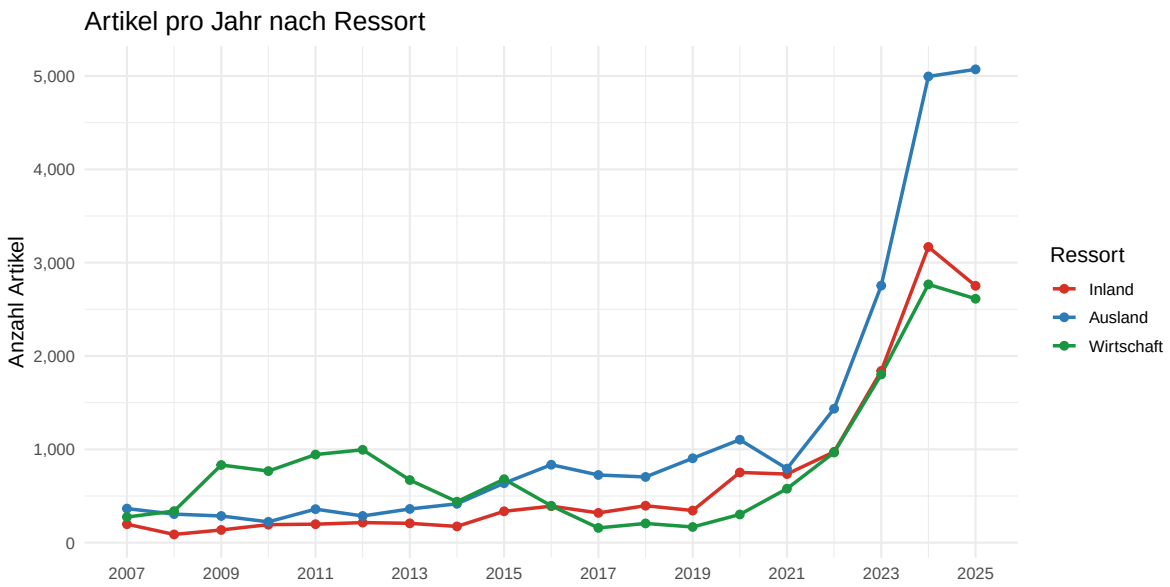


Der starke Anstieg ab 2023 dürfte vor allem auf eine **intensivierte Online-Berichterstattung** zurückgehen — tagesschau.de hat in den letzten Jahren massiv in digitale Inhalte investiert. Auch weltpolitische Großereignisse (Ukraine-Krieg, Nahostkonflikt, Bundestagswahl 2025)

treiben die Artikelzahlen nach oben. 2006 und 2026 (im Datensatz als × markiert) sind keine vollständigen Jahrgänge und sollten bei Jahresvergleichen ausgeklammert werden.

Aufgabe 28

```r
articles_slim |>
  filter(
    ressort %in% c("inland", "ausland", "wirtschaft"),
    year >= 2007, year <= 2025
  ) |>
  count(year, ressort) |>
  ggplot(aes(x = year, y = n, color = ressort)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_color_manual(
    values = c("inland" = "#d73027", "ausland" = "#2c7bb6", "wirtschaft" = "#1a9641"),
    labels = c("Inland", "Ausland", "Wirtschaft")
  ) +
  scale_y_continuous(labels = scales::comma) +
  scale_x_continuous(breaks = seq(2007, 2025, 2)) +
  labs(
    title = "Artikel pro Jahr nach Ressort",
    x     = NULL,
    y     = "Anzahl Artikel",
    color = "Ressort"
  ) +
  theme_minimal(base_size = 13)
```

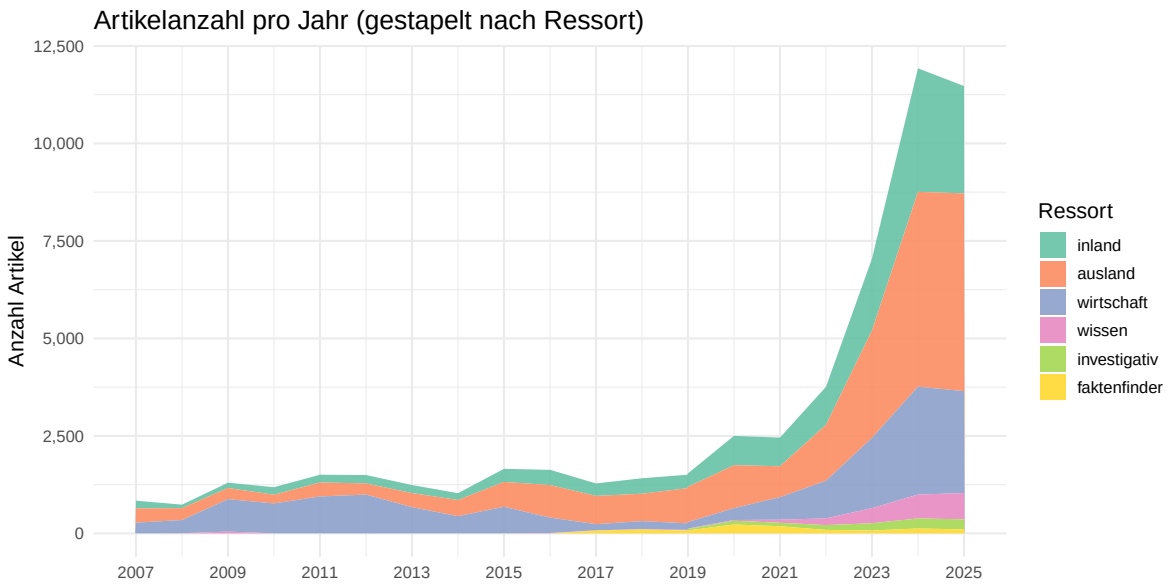


Alle drei Ressorts wachsen ab 2022/2023 stark. **`ausland`** ist seit 2023 das volumenstärkste Ressort — der Ukraine-Krieg und der Nahostkonflikt dürften dabei eine zentrale Rolle spielen. **`wirtschaft`** zeigt eine auffällige Delle in den Jahren 2016–2019, bevor es ab 2022 wieder stark anzieht.

Aufgabe 29

```
articles_slim |>
  filter(ressort %in% hauptressorts, year >= 2007, year <= 2025) |>
  count(year, ressort) |>
  mutate(ressort = fct_relevel(ressort, hauptressorts)) |>
  ggplot(aes(x = year, y = n, fill = ressort)) +
  geom_area(position = "stack", alpha = 0.9) +
  scale_fill_brewer(palette = "Set2") +
  scale_y_continuous(labels = scales::comma) +
  scale_x_continuous(breaks = seq(2007, 2025, 2)) +
  labs(
    title = "Artikelanzahl pro Jahr (gestapelt nach Ressort)",
    x     = NULL,
    y     = "Anzahl Artikel",
    fill  = "Ressort"
```

```
  ) +
  theme_minimal(base_size = 13)
```

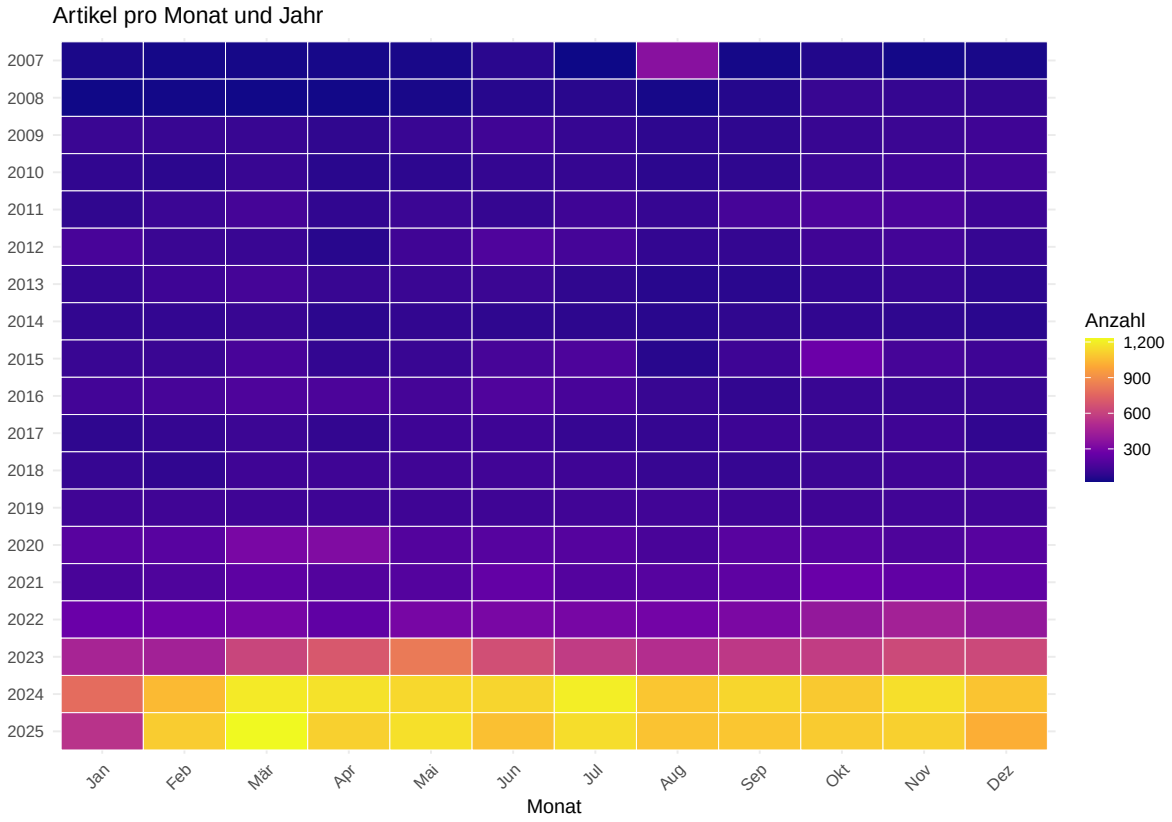


Das gestapelte Flächendiagramm zeigt im Unterschied zum Liniendiagramm das **Gesamtvolumen** über die Zeit und den **relativen Anteil** der einzelnen Ressorts. Man sieht, dass das Gesamtvolumen bis 2022 langsam wächst und danach fast explodiert. `ausland` macht den Löwenanteil des Wachstums aus, `inland` und `wirtschaft` folgen.

Aufgabe 30

```
articles_slim |>
  filter(year >= 2007, year <= 2025) |>
  count(year, month) |>
  mutate(
    year  = factor(year, levels = rev(2007:2025)),
    month = factor(month, levels = 1:12,
                   labels = c("Jan","Feb","Mär","Apr","Mai","Jun",
                              "Jul","Aug","Sep","Okt","Nov","Dez"))
  ) |>
  ggplot(aes(x = month, y = year, fill = n)) +
  geom_tile(color = "white") +
```

```r
scale_fill_viridis_c(option = "plasma", labels = scales::comma) +
labs(
  title = "Artikel pro Monat und Jahr",
  x     = "Monat",
  y     = NULL,
  fill  = "Anzahl"
) +
theme_minimal(base_size = 12) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



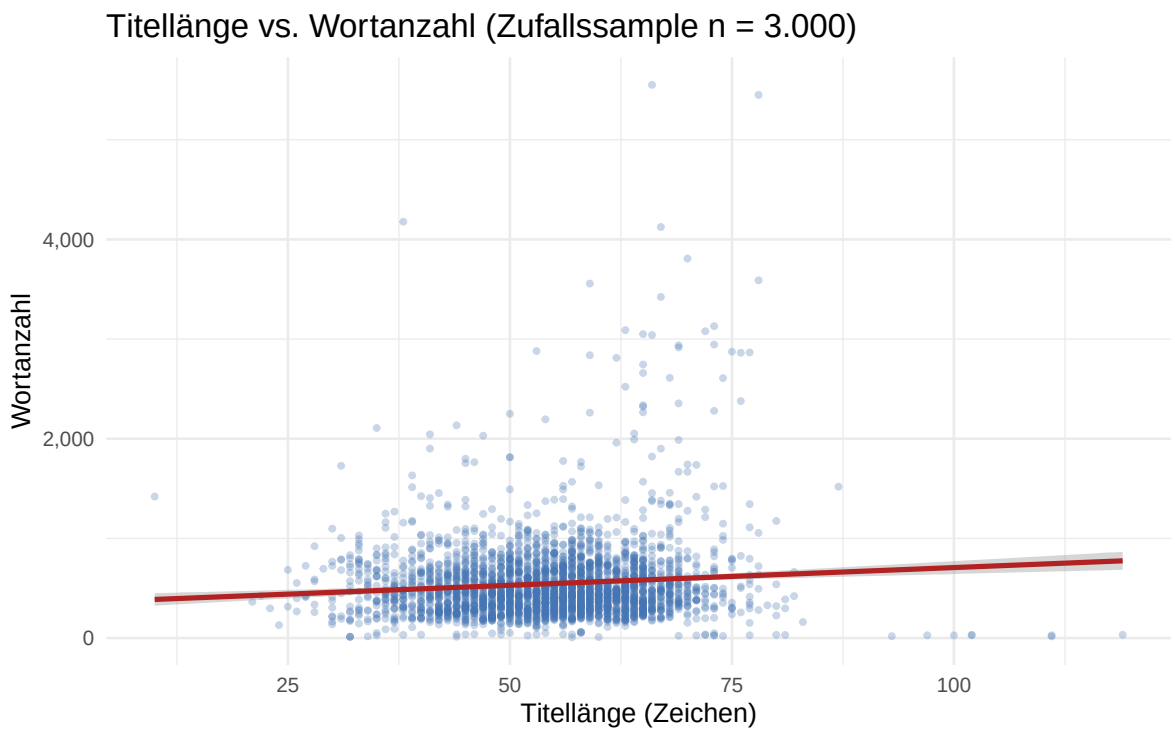
Starke saisonale Muster sind **nicht** erkennbar — die Artikelanzahl ist über das Jahr relativ gleichmäßig verteilt. Die Datendichte nimmt ab ca. **2022/2023** deutlich zu. Dieser Sprung verrät, dass der Datensatz möglicherweise nicht gleichmäßig über alle Jahre gescrapt wurde — oder dass tagesschau.de erst in jüngerer Zeit das aktuelle Volumen erreicht hat.

Teil 7: Zusammenhänge und mehrdimensionale Analysen

Aufgabe 31

```r
set.seed(42)
sample_data <- articles_slim |>
  filter(!is.na(title_length), !is.na(word_count)) |>
  slice_sample(n = 3000)

ggplot(sample_data, aes(x = title_length, y = word_count)) +
  geom_point(alpha = 0.3, size = 1, color = "#4575b4") +
  geom_smooth(method = "lm", color = "firebrick", se = TRUE) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Titellänge vs. Wortanzahl (Zufallssample n = 3.000)",
    x     = "Titellänge (Zeichen)",
    y     = "Wortanzahl"
  ) +
  theme_minimal(base_size = 13)
```



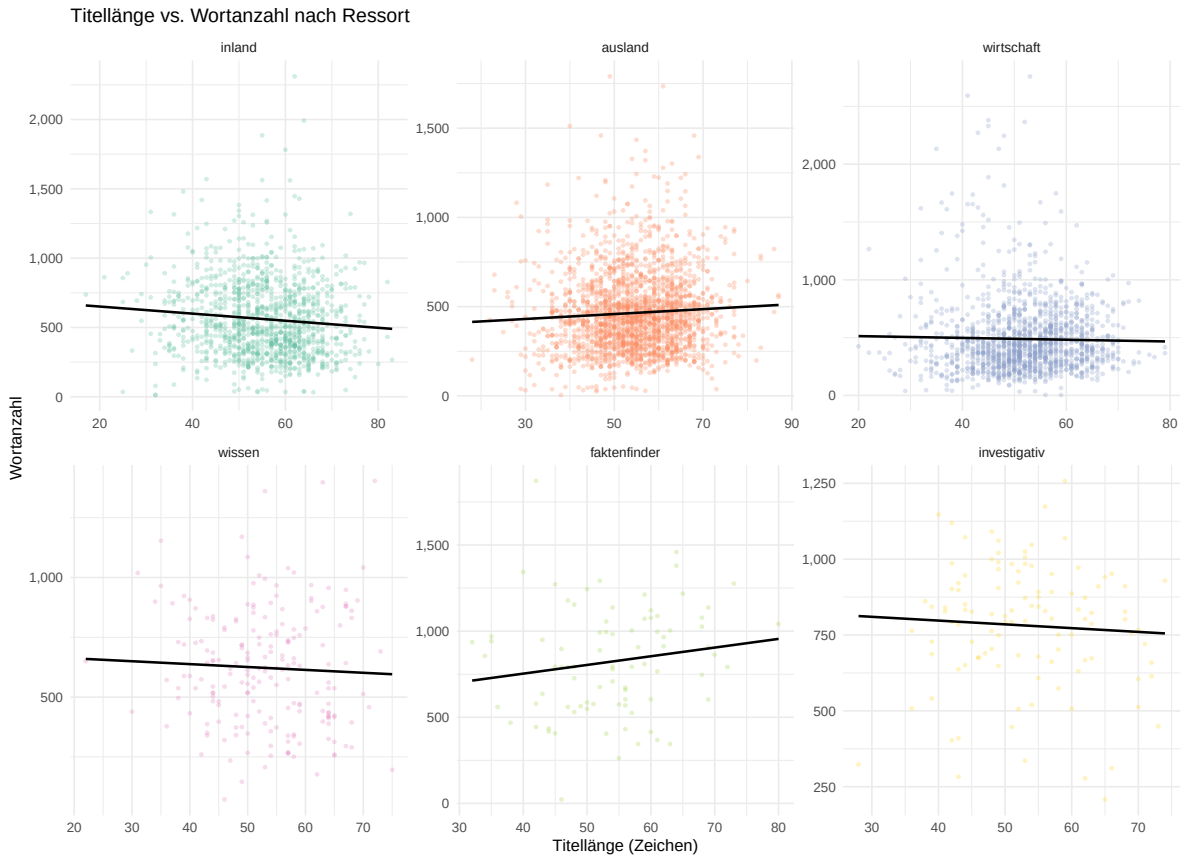
```r
cor(sample_data$title_length, sample_data$word_count, use = "complete.obs") |>
  round(3)
```

```
[1] 0.092
```

Die Korrelation beträgt **r 0,10** — ein sehr schwacher positiver Zusammenhang. Das ist überraschend: Man könnte erwarten, dass längere Artikel auch ausführlichere Titel bekommen. Tatsächlich folgen Titellängen aber eigenen redaktionellen Regeln (SEO-Optimierung, mobile Darstellung), die wenig mit der Textlänge zu tun haben. Die Trendlinie zeigt zwar eine leichte positive Steigung, die Streuung ist jedoch enorm.

Aufgabe 32

```r
set.seed(42)
articles_slim |>
  filter(ressort %in% hauptressorts, !is.na(title_length), !is.na(word_count)) |>
  slice_sample(n = 5000) |>
  ggplot(aes(x = title_length, y = word_count)) +
  geom_point(aes(color = ressort), alpha = 0.3, size = 0.8) +
  geom_smooth(method = "lm", color = "black", se = FALSE, linewidth = 0.8) +
  facet_wrap(~ ressort, scales = "free") +
  scale_color_brewer(palette = "Set2", guide = "none") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Titellänge vs. Wortanzahl nach Ressort",
    x     = "Titellänge (Zeichen)",
    y     = "Wortanzahl"
  ) +
  theme_minimal(base_size = 11)
```

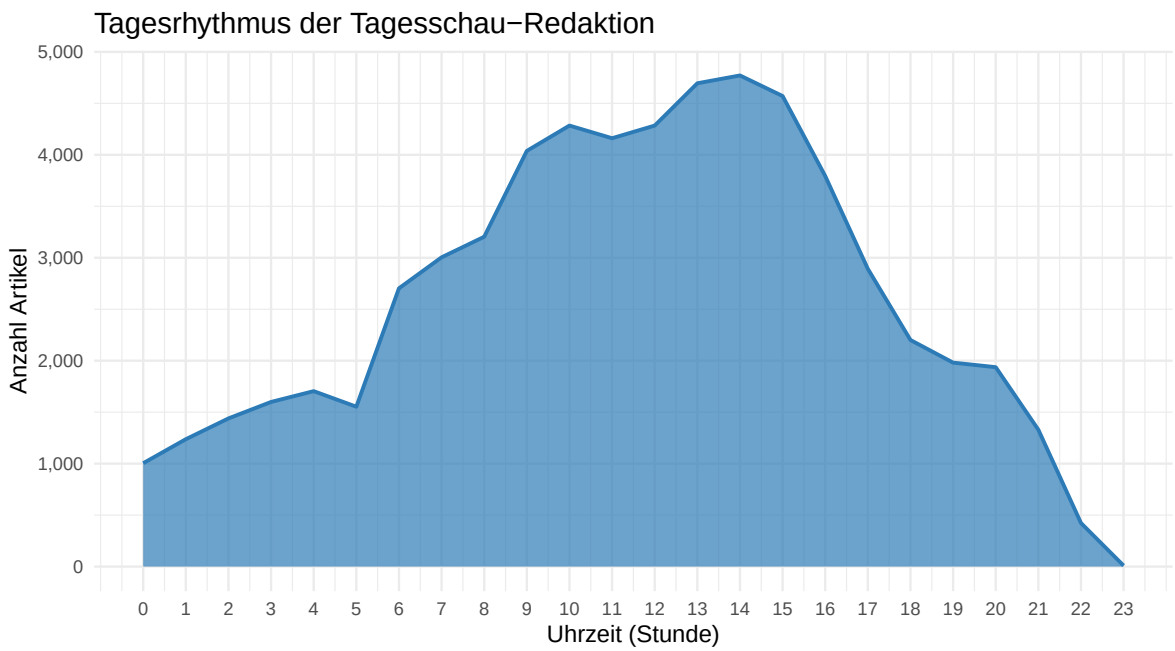


Der Zusammenhang ist in allen Ressorts schwach, aber nicht vollständig gleich. Bei **`investigativ`** und **`faktenfinder`** fällt auf, dass die Artikel im oberen Wortanzahl-Bereich konzentrierter sind und Titel eine etwas größere Variationsbreite haben — konsistent mit der inhaltlichen Ausrichtung dieser Ressorts auf tiefgründige Analysen.

Aufgabe 33

```
articles_slim |>
  mutate(hour = hour(date_time)) |>
  count(hour) |>
  ggplot(aes(x = hour, y = n)) +
  geom_area(fill = "#2c7bb6", alpha = 0.7) +
  geom_line(color = "#2c7bb6", linewidth = 1) +
  scale_x_continuous(breaks = 0:23) +
```

```r
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Tagesrhythmus der Tagesschau-Redaktion",
    x     = "Uhrzeit (Stunde)",
    y     = "Anzahl Artikel"
  ) +
  theme_minimal(base_size = 13)
```



Die Redaktion ist zwischen **13 und 15 Uhr** am produktivsten. Ab 6 Uhr morgens nimmt die Aktivität stark zu — in Übereinstimmung mit dem Beginn des journalistischen Arbeitstags. Überraschend ist die anhaltende Aktivität in den frühen Morgenstunden (0–5 Uhr), die vermutlich auf automatisierte Nachrichtenagenturen-Übernahmen oder Nachtredaktion-Schichten zurückgeht. Ab 22 Uhr bricht die Publikationsrate stark ein.

Teil 8: Datensätze verknüpfen mit Joins

Aufgabe 34

```r
ressort_meta <- tibble(
  ressort       = c("inland", "ausland", "wirtschaft", "wissen",
                    "investigativ", "faktenfinder", "newsticker",
                    "multimedia", "kommentar", "kultur"),
  ressort_label = c("Inland", "Ausland", "Wirtschaft", "Wissen",
                    "Investigativ", "Faktenfinder", "Newsticker",
                    "Multimedia", "Kommentar", "Kultur"),
  themenbereich = c("Politik", "Politik", "Wirtschaft", "Gesellschaft",
                    "Investigativ", "Faktencheck", "Aktuell",
                    "Medien", "Meinung", "Kultur")
)

ressort_meta
```

```
# A tibble: 10 x 3
   ressort      ressort_label themenbereich
   <chr>        <chr>         <chr>
 1 inland       Inland        Politik
 2 ausland      Ausland       Politik
 3 wirtschaft   Wirtschaft    Wirtschaft
 4 wissen       Wissen        Gesellschaft
 5 investigativ Investigativ  Investigativ
 6 faktenfinder Faktenfinder  Faktencheck
 7 newsticker   Newsticker    Aktuell
 8 multimedia   Multimedia    Medien
 9 kommentar    Kommentar     Meinung
10 kultur       Kultur        Kultur
```

Aufgabe 35

```r
articles_enriched <- articles_slim |>
  left_join(ressort_meta, by = "ressort")

# Artikel ohne Match
articles_enriched |>
  filter(is.na(ressort_label)) |>
  count(ressort, sort = TRUE) |>
  print(n = 15)
```

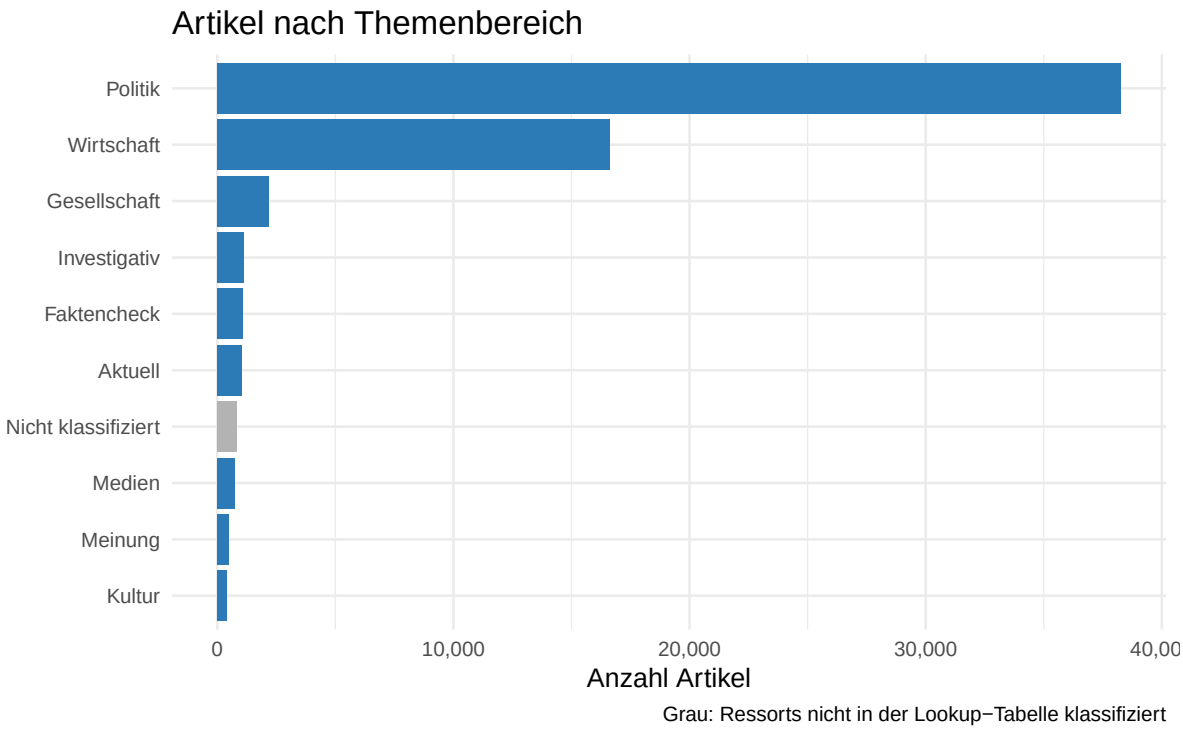
```
# A tibble: 30 x 2
   ressort                                   n
   <chr>                                 <int>
 1 Einstellungen einblenden Pfeil rechts   260
 2 europawahl                              174
 3 eilmeldung                              131
 4 nachrichten                              88
 5 sport                                    65
 6 jahresrueckblick                         20
 7 wetter                                   14
 8 schlusslicht                              9
 9 hilfe                                     8
10 news                                      8
11 spendenkonten                             8
12 politik                                   7
13 swraktuell                                5
14 wetterthema                               5
15 <NA>                                      5
# i 15 more rows
```

Ein `left_join()` behält **alle Zeilen der linken Tabelle** (`articles_slim`) — auch solche, für die in `ressort_meta` kein passender Schlüssel existiert. Diese erhalten `NA` in den hinzugefügten Spalten.

838 Artikel haben keinen Match. Darunter sind Ressorts wie `"europawahl"` (174), `"eilmeldung"` (131) und technische Artefakte.

```r
articles_enriched |>
  mutate(themenbereich = replace_na(themenbereich, "Nicht klassifiziert")) |>
  count(themenbereich, sort = TRUE) |>
  mutate(themenbereich = fct_reorder(themenbereich, n)) |>
  ggplot(aes(x = themenbereich, y = n,
             fill = themenbereich == "Nicht klassifiziert")) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "grey70", "FALSE" = "#2c7bb6"),
    guide  = "none"
  ) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title   = "Artikel nach Themenbereich",
    caption = "Grau: Ressorts nicht in der Lookup-Tabelle klassifiziert",
```

```
    x        = NULL,
    y        = "Anzahl Artikel"
  ) +
  theme_minimal(base_size = 13)
```



Ohne den Join wäre nur eine Auswertung nach den 39 einzelnen `ressort`-Werten möglich gewesen. Durch `themenbereich` ergibt sich eine semantisch sinnvolle Verdichtung auf wenige Kategorien — direkt aus der Lookup-Tabelle, ohne den Hauptdatensatz zu verändern. Das grau hervorgehobene „Nicht klassifiziert" macht zudem sofort sichtbar, wie groß der nicht abgedeckte Anteil ist.

Aufgabe 36

```
articles_inner <- articles_slim |>
  inner_join(ressort_meta, by = "ressort")

nrow(articles_inner)
```

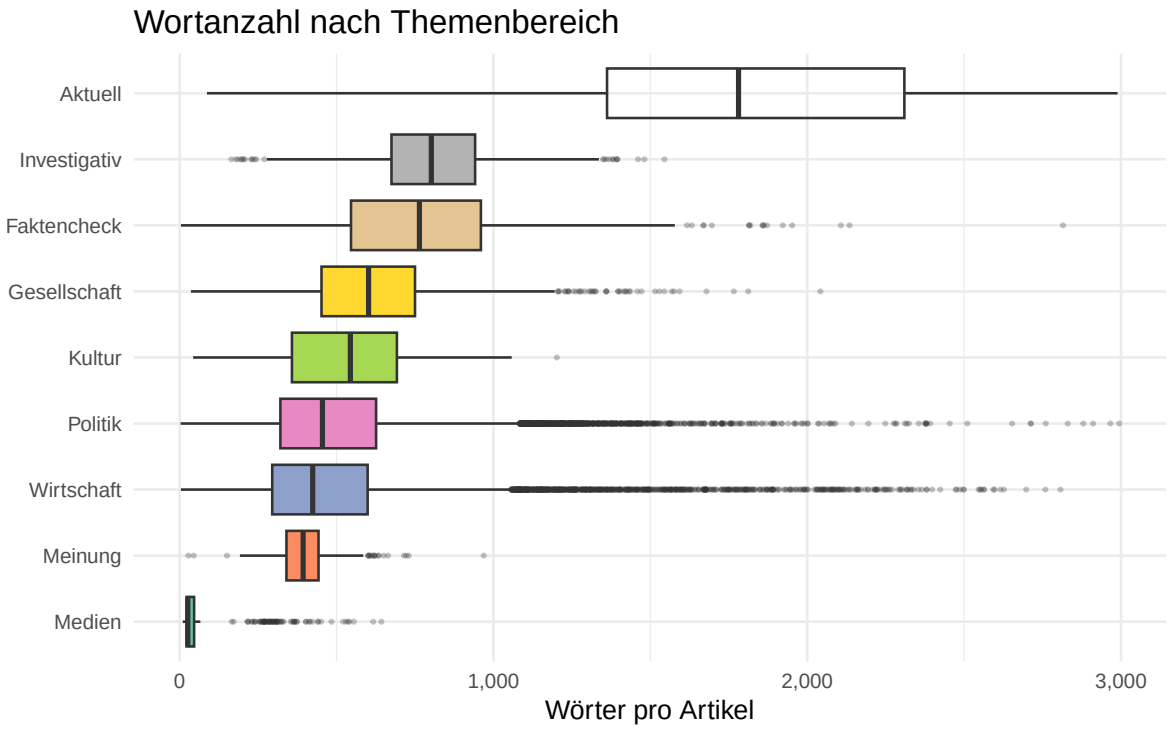
```
[1] 61979
```

```r
nrow(articles_slim) - nrow(articles_inner)
```

```
[1] 838
```

Der `inner_join()` liefert **61.979 Zeilen** — 838 weniger als der `left_join()`. Diese Zeilen werden stillschweigend fallen gelassen. Für Auswertungen, die ausschließlich `themenbereich` als Gruppierung verwenden, ist das unproblematisch. Bei Gesamtzählungen würde man jedoch 838 Artikel unbemerkt verlieren.

```r
articles_inner |>
  filter(word_count < 3000) |>
  mutate(themenbereich = fct_reorder(themenbereich, word_count, .fun = median)) |>
  ggplot(aes(x = themenbereich, y = word_count, fill = themenbereich)) +
  geom_boxplot(outlier.alpha = 0.3, outlier.size = 0.7) +
  coord_flip() +
  scale_fill_brewer(palette = "Set2", guide = "none") +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Wortanzahl nach Themenbereich",
    x     = NULL,
    y     = "Wörter pro Artikel"
  ) +
  theme_minimal(base_size = 13)
```



Der Themenbereich **Investigativ** produziert die mit Abstand längsten Artikel, gefolgt von **Faktencheck**. Beide erfordern intensive Recherche, Quellenangaben und ausführliche Erklärungen — das spiegelt sich direkt in der Wortanzahl wider. **Politik** und **Wirtschaft** haben ähnliche, niedrigere Mediane, was zur schnellen Nachrichtenlogik dieser Ressorts passt. Erst durch den Join mit `ressort_meta` wurde diese thematische Perspektive möglich: Die ursprüngliche Ressort-Variable wäre zu kleinteilig für einen sauberen Gruppenvergleich gewesen.

Aufgabe 37

```r
articles_slim |>
  anti_join(ressort_meta, by = "ressort") |>
  count(ressort, sort = TRUE) |>
  print(n = 20)
```

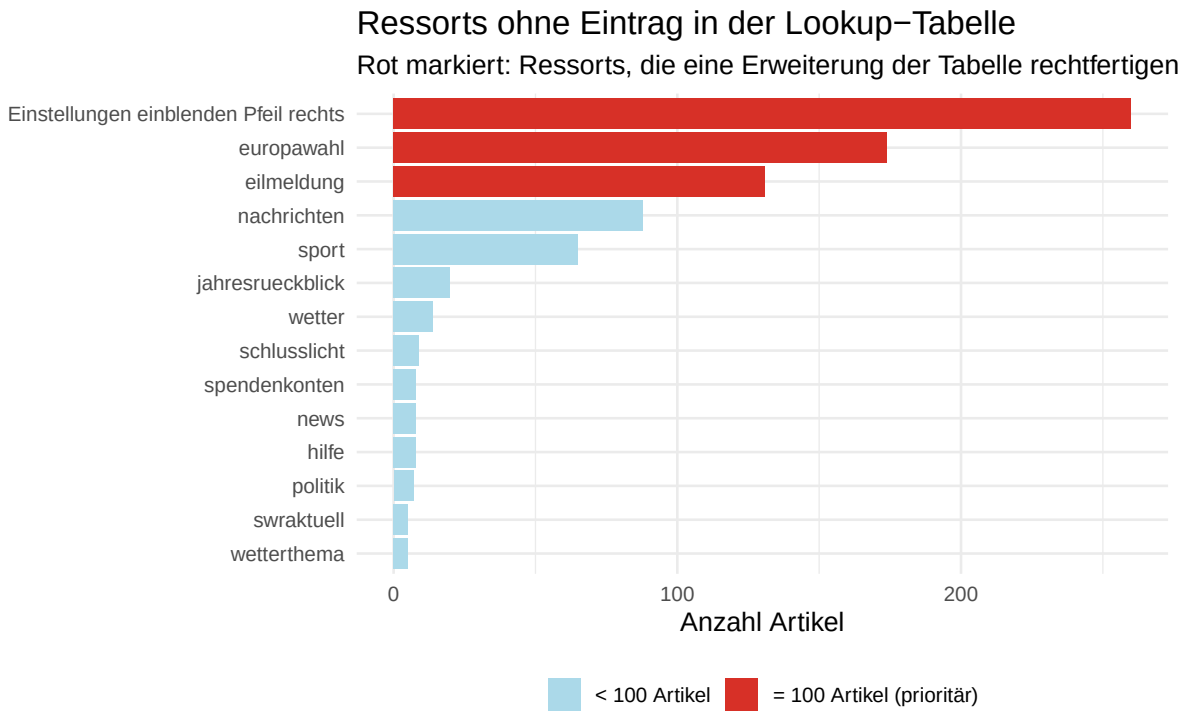
```
# A tibble: 30 x 2
   ressort                                     n
   <fct>                                     <int>
```

```
 1 Einstellungen einblenden Pfeil rechts   260
 2 europawahl                              174
 3 eilmeldung                              131
 4 nachrichten                              88
 5 sport                                    65
 6 jahresrueckblick                         20
 7 wetter                                   14
 8 schlusslicht                              9
 9 hilfe                                     8
10 news                                      8
11 spendenkonten                             8
12 politik                                   7
13 wetterthema                               5
14 swraktuell                                5
15 <NA>                                      5
16 korrespondenten                           4
17 hintergrund                               4
18 fussball                                  4
19 app                                       3
20 tutorials                                 2
# i 10 more rows
```

`anti_join()` gibt genau die Zeilen zurück, die **keinen** Match in der zweiten Tabelle finden. Das Ergebnis entspricht exakt den Zeilen, die beim `left_join()` `NA` in `ressort_label` erhalten hätten. Damit ist `anti_join()` das ideale Diagnosewerkzeug vor einem `inner_join()`: Man sieht vorab, welche Daten man verlieren würde.

```r
articles_slim |>
  anti_join(ressort_meta, by = "ressort") |>
  filter(!is.na(ressort)) |>
  count(ressort, sort = TRUE) |>
  filter(n >= 5) |>
  mutate(
    ressort    = fct_reorder(ressort, n),
    prioritaet = n >= 100
  ) |>
  ggplot(aes(x = ressort, y = n, fill = prioritaet)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "#d73027", "FALSE" = "#abd9e9"),
    labels = c("TRUE" = "  100 Artikel (prioritär)", "FALSE" = "< 100 Artikel"),
```

```
    name    = NULL
  ) +
  labs(
    title    = "Ressorts ohne Eintrag in der Lookup-Tabelle",
    subtitle = "Rot markiert: Ressorts, die eine Erweiterung der Tabelle rechtfertigen",
    x        = NULL,
    y        = "Anzahl Artikel"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```



Rot hervorgehoben sind `"europawahl"` (174 Artikel) und `"eilmeldung"` (131 Artikel) — zwei Ressorts, die thematisch klar zuordenbar wären (`"Politik"` bzw. `"Aktuell"`) und zusammen über 300 Artikel ausmachen. Eine vollständige Lookup-Tabelle sollte diese Einträge enthalten. Die hellblau markierten Ressorts sind selten genug (< 100 Artikel), um sie für die meisten Analysen unter `"Sonstiges"` zusammenzufassen.

Reflexion

Aufgabe 38

`summarize()` in Kombination mit `group_by()` **reduziert** den Datensatz auf eine Zeile pro Gruppe. Das Ergebnis ist eine neue, kompaktere Tabelle. `mutate()` in Kombination mit `group_by()` **behält alle Zeilen** und fügt lediglich eine neue Spalte hinzu, deren Wert gruppenspezifisch berechnet wird.

Beispiel für `summarize()`: Aufgabe 17 (Kenngrößen pro Ressort) — dort wollten wir eine kompakte Übersichtstabelle, keine erweiterte Einzelartikel-Liste.

Beispiel für `group_by() + mutate()`: Aufgabe 19 (ressortspezifischer Median) — dort brauchten wir den Medianwert als neue Spalte *pro Artikel*, um individuelle Artikel mit dem Ressort-Benchmark zu vergleichen. Mit `summarize()` wäre das nicht möglich gewesen, weil es den Datensatz auf Ressort-Ebene zusammengebrochen hätte.

Inhaltlich falsch wäre die Verwechslung z. B. in folgendem Fall: Will man prüfen, ob ein bestimmter Artikel über dem Durchschnitt seines Ressorts liegt, braucht man `group_by() + mutate()`. Würde man stattdessen `summarize()` verwenden, erhält man nur den Durchschnittswert pro Ressort — aber keinen Vergleich für die einzelnen Artikel.

Aufgabe 39

Drei bemerkenswerte Beobachtungen (Beispielantwort):

1. **Die Titellänge korreliert kaum mit der Artikellänge (r 0,10).** Das ist kontraintuitiv. Eine mögliche Erklärung: Titel folgen redaktionellen und technischen Konventionen (z. B. maximal 60–70 Zeichen für Google-Suche), die von der inhaltlichen Tiefe des Artikels unabhängig sind.
2. **Fast 27 % aller Artikel haben keinen Autor.** Das zeigt, wie stark tagesschau.de auf kollektiv-redaktionelle Arbeit setzt. Der Anteil namentlicher Autoren steigt erst ab ca. 2016 merklich — möglicherweise ein Zeichen für eine gewandelte Autorenkultur im digitalen Journalismus.
3. **Zwischen ca. 0 und 6 Uhr werden erstaunlich viele Artikel publiziert.** Dieser „stille" Aktivitätszeitraum deutet auf Nachtredaktionen, automatische Agenturübernahmen oder internationale Zeitzonenkorrekturen hin — ein Detail, das im normalen Medienkonsum unsichtbar bleibt.

Aufgabe 40

Visualisierungs-form	Fragetyp	Variablentyp(en)	Typischer Einsatz
Histogramm	univariat	metrisch	Verteilung einer Zahl-Variable
Balkendiagramm	univariat	kategorial	Häufigkeiten von Kategorien
Boxplot	bivariat	kategorial × metrisch	Gruppenvergleiche einer Zahl-Variable
Boxplot + Jitter	bivariat	kategorial × metrisch	Wie Boxplot, aber mit Rohdatenpunkten
Ridgeline-Plot	bivariat	kategorial × metrisch	Verteilungsform über mehrere Gruppen
Heatmap	bivariat	kategorial × kategorial	Kreuztabelle als Farbe
Liniendiagramm	bivariat	zeitlich × metrisch	Zeitreihen, Trends
Flächendiagramm	bi-/trivariat	zeitlich × metrisch (+ kat.)	Trends mit Gesamtvolumen oder Anteilen
Scatter-Plot	bivariat	metrisch × metrisch	Zusammenhänge zweier Zahl-Variablen

Bei Überschneidungen entscheidet meist die **Informationspriorität**: Geht es um den groben Vergleich (Boxplot), die genaue Verteilungsform (Ridgeline), oder die rohen Datenpunkte (Jitter)? Die Datenmenge spielt ebenfalls eine Rolle: Jitter bei 25.000 Punkten produziert schwarze Wolken; hier ist ein Boxplot oder Ridgeline lesbarer.

Aufgabe 41

Mindestens vier konkrete Qualitätsprobleme im Datensatz:

1. **Unvollständige Jahrgänge (2006, 2026):** Jahresvergleiche ohne Berücksichtigung dieser Jahrgänge verzerren Trends. Lösung: Explizites Filtern auf vollständige Jahrgänge; im Methodenteil dokumentieren.
2. **Technische Artefakte als Ressorts:** Der Wert `"Einstellungen einblenden Pfeil rechts"` ist offensichtlich kein Ressort, sondern ein Scraping-Fehler. Lösung: Vorab bereinigen; betroffene Zeilen aus inhaltlichen Analysen ausschließen.
3. **Duplikate in den Tags (`"FAKTENFINDER"` vs. `"#FAKTENFINDER"`):** Eine uneinheitliche Kodierung führt zu falschen Häufigkeitsauszählungen. Lösung: Standardisierung vor der Analyse (Präfixe entfernen, Groß-/Kleinschreibung vereinheitlichen).

4. **Strukturbruch über die Zeit:** Das Datensatzvolumen nimmt ab 2023 stark zu. Das könnte auf eine veränderte Scrapin-Strategie, eine Erweiterung des Webangebots oder eine redaktionelle Umorientierung zurückgehen. Vergleiche zwischen frühen und späten Jahren sind daher möglicherweise nicht gleichwertig. Lösung: Volumenveränderungen transparent machen; zeitstratifizierte Analysen bevorzugen.

5. *(Bonus)* **Fehlende `word_count`-Werte (367 NA):** Diese konzentrieren sich auf bestimmte Artikel, was auf systematische Datenlücken hindeuten kann. Lösung: Vor jeder Analyse, die `word_count` verwendet, explizit auf Vollständigkeit prüfen und `na.rm = TRUE` konsistent einsetzen.